

Planning

What is Classical Planning?

- Classical planning = finding a sequence of deterministic actions from a fully-observed initial state to a goal state under closed-world assumptions.
- **Input:** initial state (fluents), action schemas, goal condition.
- **Output:** a plan = ordered actions that, when executed in the model, yield a state that satisfies the goal.
- **Assumptions:** deterministic, fully observable, static world (no exogenous events), discrete time steps.

States as Sets of Fluents

A state is a set (or conjunction) of ground fluents (function-free atomic formulas).

- **Examples of allowed fluents:** $\text{At}(\text{Truck1}, \text{Melbourne})$, $\text{In}(\text{C1}, \text{P1})$, $\text{Clear}(\text{A})$.
- **Not allowed:** non-ground terms $\text{At}(x,y)$, negations as top-level $\neg\text{Poor}$, or terms containing functions $\text{At}(\text{Father}(\text{Fred}), \text{Sydney})$.

Action Schemas: What and Why

An action schema compactly specifies many ground actions by using variables — it is a lifted (first-order) description.

Structure: Action(Name(vars), PRECOND: ..., EFFECT: ...)

where PRECOND and EFFECT are conjunctions of literals.

Example: Action(
Fly(p, from, to),

PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$

)

Applicability of Actions

An action is applicable in state s if s satisfies its precondition: $s \models \text{PRECOND}(a)$.

- Set semantics test: Every positive literal in PRECOND is in s , and every negated literal is not in s .
- Complexity note: If action has v variables and domain has k objects, worst-case instantiations to check $\approx O(k^v)$ to find all applicable ground actions.

RESULT Function

- $\text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a)$

where $\text{DEL}(a)$ = negative **effect literals**, $\text{ADD}(a)$ = positive **effect literals**.

- **Example:** Executing $\text{Fly}(P1, \text{SFO}, \text{JFK})$ in a state containing $\text{At}(P1, \text{SFO})$ removes $\text{At}(P1, \text{SFO})$ and adds $\text{At}(P1, \text{JFK})$.

Important rule: EFFECT variables must appear in PRECOND so that DEL/ADD are ground when applied.

Modeling: Initial State and Goal

- Initial state: Conjunction of ground atoms. Use closed-world assumption.
- Goal: Conjunction (or set) of literals possibly containing variables (treated existentially).
- Example: $\text{Goal}(\text{At}(p, \text{SFO}) \wedge \text{Plane}(p))$ means “there exists a plane at SFO.”

Example-Result Function

- Action(Fly(p, from, to),
PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$
EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$)
- $s_0 = \{ \text{At}(P1, \text{SFO}), \text{At}(P2, \text{JFK}), \text{Plane}(P1), \text{Plane}(P2), \text{Airport}(\text{SFO}), \text{Airport}(\text{JFK}) \}$

$a = \text{Fly}(P1, \text{SFO}, \text{JFK})$

$\text{PRECOND}(a) = \{ \text{At}(P1, \text{SFO}), \text{Plane}(P1), \text{Airport}(\text{SFO}), \text{Airport}(\text{JFK}) \}$

•EFFECT(a) contains positive and negative literals:

$\text{DEL}(a) = \{ \text{At}(P1, \text{SFO}) \}$ (negative effect)

$\text{ADD}(a) = \{ \text{At}(P1, \text{JFK}) \}$ (positive effect)

-> s_0 - $\text{DEL}(a) = \{ \text{At}(P2, \text{JFK}), \text{Plane}(P1), \text{Plane}(P2), \text{Airport}(\text{SFO}), \text{Airport}(\text{JFK}) \}$

-> $\text{RESULT}(s_0, a) = \{ \text{At}(P2, \text{JFK}), \text{Plane}(P1), \text{Plane}(P2), \text{Airport}(\text{SFO}), \text{Airport}(\text{JFK}), \text{At}(P1, \text{JFK}) \}$

Example Walkthrough: Air Cargo

- Domain predicates: $At(x, a)$, $In(c, p)$, $Cargo(c)$, $Plane(p)$, $Airport(a)$.
- Init (simple): $At(C1, SFO)$, $At(C2, JFK)$, $At(P1, SFO)$, $At(P2, JFK)$
- Goal: $At(C1, JFK) \wedge At(C2, SFO)$
- Plan (one valid): Load C1 onto P1, Fly P1 $SFO \rightarrow JFK$, Unload C1, then symmetric steps for C2.

Action schemas

- Action: Load(c, p, a)
 - PRECOND: $\text{At}(c, a) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$
 - EFFECT: $\neg \text{At}(c, a) \wedge \text{In}(c, p)$
-
- Action: Unload(c, p, a)
 - PRECOND: $\text{In}(c, p) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$
 - EFFECT: $\text{At}(c, a) \wedge \neg \text{In}(c, p)$
-
- Action: Fly(p, from, to)
 - PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to}) \wedge (\text{from} \neq \text{to})$
 - EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$

Ordered action

- a1 = Load(C1, P1, SFO)
- a2 = Fly(P1, SFO, JFK)
- a3 = Unload(C1, P1, JFK)
- a4 = Load(C2, P2, JFK)
- a5 = Fly(P2, JFK, SFO)
- a6 = Unload(C2, P2, SFO)

- $s_0 = \{ \text{At}(\text{C1}, \text{SFO}), \text{At}(\text{C2}, \text{JFK}), \text{At}(\text{P1}, \text{SFO}), \text{At}(\text{P2}, \text{JFK}), \text{Cargo}(\text{C1}), \text{Cargo}(\text{C2}), \text{Plane}(\text{P1}), \text{Plane}(\text{P2}), \text{Airport}(\text{SFO}), \text{Airport}(\text{JFK}) \}$

$$s_1 = (s_0 - \{ \text{At}(\text{C1}, \text{SFO}) \}) \cup \{ \text{In}(\text{C1}, \text{P1}) \}$$
$$s_1 = \{ \text{At}(\text{C2}, \text{JFK}), \text{At}(\text{P1}, \text{SFO}), \text{At}(\text{P2}, \text{JFK}), \text{In}(\text{C1}, \text{P1}), \text{Cargo}(\text{C1}), \text{Cargo}(\text{C2}), \text{Plane}(\text{P1}), \text{Plane}(\text{P2}), \text{Airport}(\text{SFO}), \text{Airport}(\text{JFK}) \}$$
$$s_2 = (s_1 - \{ \text{At}(\text{P1}, \text{SFO}) \}) \cup \{ \text{At}(\text{P1}, \text{JFK}) \}$$
$$s_2 = \{ \text{At}(\text{C2}, \text{JFK}), \text{At}(\text{P1}, \text{JFK}), \text{At}(\text{P2}, \text{JFK}), \text{In}(\text{C1}, \text{P1}), \text{Cargo}(\text{C1}), \text{Cargo}(\text{C2}), \text{Plane}(\text{P1}), \text{Plane}(\text{P2}), \text{Airport}(\text{SFO}), \text{Airport}(\text{JFK}) \}$$
$$s_3 = (s_2 - \{ \text{In}(\text{C1}, \text{P1}) \}) \cup \{ \text{At}(\text{C1}, \text{JFK}) \}$$
$$s_3 = \{$$
$$\text{At}(\text{C1}, \text{JFK}), \text{At}(\text{C2}, \text{JFK}), \text{At}(\text{P1}, \text{JFK}), \text{At}(\text{P2}, \text{JFK}),$$
$$\text{Cargo}(\text{C1}), \text{Cargo}(\text{C2}), \text{Plane}(\text{P1}), \text{Plane}(\text{P2}), \text{Airport}(\text{SFO}), \text{Airport}(\text{JFK})$$
$$\}$$

- $s_4 = (s_3 - \{At(C2,JFK)\}) \cup \{In(C2,P2)\}$
- $s_4 = \{ At(C1,JFK), At(P1,JFK), At(P2,JFK), In(C2,P2), Cargo(C1), Cargo(C2), Plane(P1), Plane(P2), Airport(SFO), Airport(JFK) \}$
- $s_5 = (s_4 - \{At(P2,JFK)\}) \cup \{At(P2,SFO)\}$
- $s_5 = \{ At(C1,JFK), At(P1,JFK), At(P2,SFO), In(C2,P2), Cargo(C1), Cargo(C2), Plane(P1), Plane(P2), Airport(SFO), Airport(JFK) \}$
- $s_6 = (s_5 - \{In(C2,P2)\}) \cup \{At(C2,SFO)\}$
- $s_6 = \{ At(C1,JFK), At(C2,SFO), At(P1,JFK), At(P2,SFO), Cargo(C1), Cargo(C2), Plane(P1), Plane(P2), Airport(SFO), Airport(JFK) \}$

Example Walkthrough: Blocks World

- Predicates: $\text{On}(b,x)$, $\text{Clear}(x)$, $\text{Block}(b)$, Table .
- Init: $\text{On}(A, \text{Table}) \wedge \text{On}(B, \text{Table}) \wedge \text{On}(C, A) \wedge \text{Clear}(B) \wedge \text{Clear}(C)$
Goal: $\text{On}(A,B) \wedge \text{On}(B,C)$
- Actions: $\text{Move}(b,x,y)$ and $\text{MoveToTable}(b,x)$ with preconditions ensuring Clear and Block where needed.
- Plan: $[\text{MoveToTable}(C, A), \text{Move}(B, \text{Table}, C), \text{Move}(A, \text{Table}, B)]$ (one minimal plan).

From Planning Problem to Search Problem

A planning problem (Init, Actions, Goal) is a search problem: nodes = states (or state descriptions), edges = $\text{RESULT}(s,a)$ for applicable actions. Search algorithms traverse this graph to reach a goal node.

- **Two canonical directions:**
- **Forward (progression):** expand from Init using $\text{ACTIONS}(s)$ and $\text{RESULT}(s,a)$.
- **Backward (regression):** start with Goal descriptions and regress actions to find predecessor descriptions that could lead to Goal.

Forward (Progression) Search

- Definition: Start at the initial state s_0 . At each node s , generate all applicable ground actions $a \in \text{ACTIONS}(s)$, compute $\text{RESULT}(s,a)$, and continue until a state that satisfies the goal is reached.
- Algorithmic options: breadth-first, depth-first, A^* /heuristic search, and local search variants.

Why forward search can be hard

- Two main causes of difficulty: Irrelevant actions / huge action schemas: schemas like Buy(isbn) represent billions of ground actions if naively instantiated. Forward search may need to consider many irrelevant ground actions.
- Large branching factors and state spaces: realistic domains (e.g., air cargo with many planes & packages) can have thousands of actions applicable in a single state → enormous search trees.
- Concrete intuition: with ~ 2000 actions per state and depth 4, naive forward search explores $\sim 2000^4$ nodes.

Backward (Regression) Search

- Start from the goal description g (a conjunction of literals describing a set of states). For a candidate ground action a whose effects contribute to g , compute the regressed description g' that must have been true before a executed. Continue regressing until g' is entailed by Init .
- Regression formula (ground): $g' = (g - \text{ADD}(a)) \cup \text{PRECOND}(a)$. That is, remove from g any fluents that a would have added, and add a 's preconditions.

Intuition behind the regression formula

- Why $(g - \text{ADD}(a))$?

fluents in $\text{ADD}(a)$ need not have been true before a because a establishes them.

- Why $\cup \text{PRECOND}(a)$?

because a could only have executed if its preconditions were true before.

- Why $\text{DEL}(a)$ is absent?

we cannot infer whether deleted fluents were true before the action; regression is conservative and does not assume their values.

Regression with variables (lifted regression)

- Lifted actions: regress using standardized action schemas with variables (first-order). Use unification rather than enumerating ground actions.
- Most general unifier (MGU): when goal literal g_i unifies with action effect e_j under substitution θ , regress using $\text{SUBST}(\theta, A)$ producing a lifted predecessor description that covers many ground predecessors.
- Benefit: dramatically reduces branching (e.g., $\text{Unload}(C2, p, \text{SFO})$ covers all planes p).

Choosing relevant actions

- Relevance criteria (technical): An action is relevant to g if at least one of its effect literals unifies with a literal in g . Additionally, no effect of the action may negate a literal in g (otherwise it cannot be the final step achieving g).
- Heuristic choice: use the action with the most general substitution (avoid unnecessary grounding).
- Clarify with a short counterexample: If g contains C and action a has effect $A \wedge B \wedge \neg C$, then a may seem helpful (A and B) but cannot be final as it deletes C . Thus it is not relevant as last step for g .

Worked examples

- Buy ISBN example: Goal $\text{Own}(0136042597)$ $\rightarrow$ unify with $\text{Buy}(i)$ effect $\rightarrow$ regress to $\text{ISBN}(0136042597)$ $\rightarrow$ check Init.
- Air cargo (regression): Goal $\text{At}(C2, \text{SFO})$ $\rightarrow$ regress over Unload $\rightarrow$ need $\text{In}(C2, p) \wedge \text{At}(p, \text{SFO})$ $\rightarrow$ regress $\text{In}(C2, p)$ via Load $\rightarrow$ need $\text{At}(C2, \text{loc}) \wedge \text{At}(p, \text{loc})$ etc. Show short chain to Init.

Problem 1 — Blocks world (example setup)

Initial state (call it s0)

On(A, Table)

On(B, Table)

On(C, A)

Clear(B)

Clear(C)

Block(A), Block(B), Block(C)

(So C sits on A; A and B are on the table; B and C are clear.)

Goal

On(A, B)

On(B, C)

A correct (optimal) plan is length 3, for example:

MoveToTable(C, A) — move C from A to Table (now A becomes clear)

Move(B, Table, C) — move B onto C

Move(A, Table, B) — move A onto B

A — Ignore-Preconditions heuristic

Drop all action preconditions. Any action that has an ADD effect that contributes to a goal literal can be applied immediately from any state.

Naive version (count unsatisfied goals):

- Goal literals: $\text{On}(A,B)$ and $\text{On}(B,C)$.
- In s_0 , neither is true $\rightarrow$ number unsatisfied = **2**.
So $h_{\text{ignorePrecond_naive}}(s_0) = 2$.

Set-cover refinement (more accurate for the same relaxation):

- Universe $U = \{\text{On}(A,B), \text{On}(B,C)\}$.
- Enumerate actions (ground actions) whose ADD lists include some goal literal:
 - $\text{Move}(A, \text{Table}, B)$ adds $\text{On}(A,B)$ (also adds $\text{Clear}(\text{Table})$ but we only care about goal literals).
 - $\text{Move}(B, \text{Table}, C)$ adds $\text{On}(B,C)$.
 - (Other moves could add those literals too if different source/target present, but here those two are the natural ones.)
- We need the smallest number of actions whose effects cover U .
 - $\{\text{Move}(A,\text{Table},B), \text{Move}(B,\text{Table},C)\}$ covers both $\rightarrow$ size = **2**.
- The set-cover optimum = 2, so the refined ignore-preconditions heuristic gives **2**.

B — Ignore-Delete-Lists (delete relaxation)

- **Idea** : Remove all delete effects from every action. Effects that add fluents remain; nothing ever becomes false. Preconditions still must be satisfied (unlike ignore-preconditions).
- **Compute a relaxed plan for s0:**
- We must respect preconditions — an action is only allowed if its preconditions (positive literals) hold — but because delete lists are ignored, performing an action never disables other facts.
- Start s0 has: On(C,A), Clear(C), Clear(B), On(A,Table), On(B,Table).
- Goal: On(A,B) and On(B,C).
- Consider necessary steps:
- To make On(A,B) we need to move A onto B: Move(A, X, B). That requires Clear(A) and Clear(B) and On(A,X) for some X. Initially Clear(A) is **false** (because C is on A), so we must first free A.
- To free A, move C off A, e.g. MoveToTable(C, A) — this requires On(C,A) and Clear(C) which are true. After this action (even in real world), A becomes clear. In delete-relaxation we do not remove other facts, so the fact that Clear(B) stays true (even if later an action would normally make it false) means concurrency is easier.
- After MoveToTable(C,A), we can do Move(A, Table, B) to get On(A,B). And now we can do Move(B, Table, C) to get On(B,C).

- So one relaxed plan is:
- MoveToTable(C,A)
- Move(A,Table,B)
- Move(B,Table,C)
- Length = **3**.
- Could there be a shorter relaxed plan? No — because A is not clear in the initial state, and preconditions still require clearing it first; thus at least one action is needed to clear A. Also at least two actions are needed to create the two goal On facts unless a single action adds both goal facts — no such action exists. So **3** is minimal for this relaxed model.
- Therefore $h_ignoreDelete(s_0) = 3$.